

ITEM 01

Nível de Dificuldade: Fácil

Objetos de Conhecimento: 8.3 Boas práticas

Capacidade: CT01 – Reconhecer requisitos de qualidade, integridade, usabilidade e segurança da informação.

Contexto: Durante a manutenção de um sistema de e-commerce legado, um desenvolvedor identifica que diversas variáveis globais estão sendo sobrescritas acidentalmente em diferentes partes do código devido ao uso de uma palavra-chave antiga de declaração que ignora o escopo de bloco.

Comando: Qual palavra-chave do JavaScript moderno deve ser utilizada para declarar variáveis que permitem reatribuição, mas que ficam restritas ao escopo de bloco onde foram criadas?

Alternativas:

- ☐ A) var
- ☐ B) let
- ☐ C) const
- ☐ D) function
- ☐ E) global

Gabarito Sinalizado: B

Justificativa: A palavra-chave **let** foi introduzida no JavaScript moderno para permitir a criação de variáveis mutáveis com escopo de bloco, evitando o vazamento de dados comum ao **var**.

Justificativa dos Distratores:

- A) O **var** possui escopo de função ou global, causando o problema citado no contexto.
 - C) O **const** impede a reatribuição do valor, o que não atende à necessidade de variáveis mutáveis.
 - D) **function** declara blocos de código, não variáveis.
 - E) **global** não é uma palavra-chave de declaração de variável no JavaScript.
-

ITEM 02

Nível de Dificuldade: Fácil

Objetos de Conhecimento: 8.1 Tipos

Capacidade: CT01 – Reconhecer requisitos de qualidade, integridade, usabilidade e segurança da informação.

Contexto: No JavaScript, existem diferentes tipos de valores que representam a ausência de dados ou estados não definidos. Um desenvolvedor precisa atribuir intencionalmente um valor vazio a uma variável de usuário.

Comando: Qual valor deve ser atribuído para indicar a ausência proposital de qualquer valor de objeto?

Alternativas:

- ☐ A) undefined
- ☐ B) NaN
- ☐ C) false
- ☐ D) null
- ☐ E) empty

Gabarito Sinalizado: D

Justificativa: O valor **null** é um "valor proposital" que o desenvolvedor atribui para dizer que a variável está intencionalmente vazia no momento.

Justificativa dos Distratores:

- A) **undefined** é atribuído automaticamente pelo sistema para indicar que a variável existe mas ainda não recebeu valor.
 - B) **NaN** significa "Not a Number", usado para representar resultados matemáticos inválidos.
 - C) **false** é um valor booleano lógico, não um indicador de vazio.
 - E) **empty** não é uma palavra-chave reservada para tipos de dados no JavaScript.
-

ITEM 03

Nível de Dificuldade: Médio

Objetos de Conhecimento: 8.3 Boas práticas

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Um desenvolvedor precisa criar uma função para calcular descontos em um carrinho de compras. Ele busca reduzir a profundidade do código (indentação) e evitar o uso de múltiplos blocos **if/else** aninhados para melhorar a legibilidade e a manutenção.

Comando: Qual técnica de programação utiliza retornos antecipados para tratar condições de erro ou exceções logo no início da função?

Alternativas:

- ☐ A) Callback Hell
- ☐ B) Pirâmide de Decisões
- ☐ C) Early Return
- ☐ D) Switch Case
- ☐ E) Hoisting

Gabarito Sinalizado: C

Justificativa: O *Early Return* (ou Cláusulas de Guarda) foca em limpar o caminho da função, tratando erros primeiro e saindo antecipadamente, o que reduz a carga cognitiva e a profundidade do código.

Justificativa dos Distratores:

- A) *Callback Hell* refere-se ao aninhamento excessivo de funções assíncronas, o que prejudica a leitura.
 - B) A "Opção Pirâmide" é justamente o que se deseja evitar ao aplicar o SRP e o Early Return.
 - D) O **switch** organiza múltiplos caminhos, mas não foca na saída antecipada por erro.
 - E) *Hoisting* é o comportamento de elevação de declarações no JS, não uma técnica de limpeza de fluxo.
-

ITEM 04

Nível de Dificuldade: Médio

Objetos de Conhecimento: 8.4 Bibliotecas e APIs

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Uma aplicação de vendas precisa transformar um array bruto de objetos de produtos em um novo array contendo apenas os preços finais calculados, sem alterar a lista original.

Comando: Qual método de array é utilizado para transformar todos os elementos, gerando uma nova coleção do mesmo tamanho?

Alternativas:

- ☐ A) filter
- ☐ B) reduce
- ☐ C) splice
- ☐ D) map
- ☐ E) forEach

Gabarito Sinalizado: D

Justificativa: O método **map** transforma cada item de um array e retorna um novo array com os resultados das transformações, mantendo a imutabilidade do original.

Justificativa dos Distratores:

- A) O **filter** cria um novo array apenas com os itens que atendem a uma condição, podendo mudar o tamanho da lista.
 - B) O **reduce** condensa os dados em um único valor final.
 - C) O **splice** modifica o array original, removendo ou adicionando itens.
 - E) O **forEach** apenas itera para causar efeitos colaterais e retorna **undefined**.
-

ITEM 05

Nível de Dificuldade: Médio

Objetos de Conhecimento: 6 Design de interface para interação de subsistemas

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Para implementar um sistema de "Modo Escuro" em um Dashboard, o programador decide alternar a presença de uma classe CSS no elemento **<body>** toda vez que um botão for acionado.

Comando: Qual método do objeto **classList** é o mais eficiente para adicionar uma classe se ela não existir e removê-la caso já esteja presente?

Alternativas:

- ☐ A) add()

- ☐ B) remove()
- ☐ C) contains()
- ☐ D) toggle()
- ☐ E) replace()

Gabarito Sinalizado: D

Justificativa: O método `toggle()` atua como um interruptor inteligente, eliminando a necessidade de estruturas condicionais manuais para verificar a existência da classe antes da alteração.

Justificativa dos Distratores:

- A) `add()` apenas adiciona a classe, exigindo lógica extra para remoção.
 - B) `remove()` apenas retira a classe.
 - C) `contains()` apenas verifica a existência da classe e retorna um booleano.
 - E) `replace()` substitui uma classe por outra, não servindo para alternar um estado único.
-

ITEM 06

Nível de Dificuldade: Médio

Objetos de Conhecimento: 8.4 Bibliotecas e APIs

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Ao gerenciar operações assíncronas modernas, um programador utiliza objetos que representam um "vale" ou recibo para um valor que estará disponível no futuro.

Comando: Como se chama o objeto que possui os estados Pendente (*Pending*), Resolvida (*Fulfilled*) e Rejeitada (*Rejected*)?

Alternativas:

- ☐ A) Callback
- ☐ B) Promise
- ☐ C) JSON
- ☐ D) Event Loop
- ☐ E) Async Function

Gabarito Sinalizado: B

Justificativa: Uma *Promise* é um objeto que representa o sucesso ou a falha eventual de uma operação assíncrona, organizando o fluxo de dados através de seus estados internos.

Justificativa dos Distratores:

- A) Callbacks são o modelo antigo que causava o "Callback Hell", sem estados formais de objeto.
 - C) JSON é um formato de texto para transporte de dados.
 - D) *Event Loop* é o mecanismo de gerenciamento de fila de tarefas do navegador.
 - E) *Async Function* é a sintaxe para lidar com Promises, não o objeto de estado em si.
-

ITEM 07

Nível de Dificuldade: Médio

Objetos de Conhecimento: 8.4 Bibliotecas e APIs

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Para integrar o front-end com um serviço externo de consulta de endereços (ViaCEP), o sistema precisa transformar um objeto JavaScript em uma string de texto puro para ser enviada pela rede.

Comando: Qual método deve ser utilizado para converter um objeto JavaScript em uma string no formato JSON?

Alternativas:

- ☐ A) JSON.parse()
- ☐ B) JSON.stringify()
- ☐ C) Fetch()
- ☐ D) response.json()
- ☐ E) Object.keys()

Gabarito Sinalizado: B

Justificativa: O método **JSON.stringify()** "empacota" objetos da memória em texto puro JSON para transporte via protocolo HTTP.

Justificativa dos Distratores:

- A) **JSON.parse()** faz o processo inverso: converte texto recebido em objeto manipulável.
 - C) **Fetch()** é a função de requisição, não de conversão de formato.
 - D) **response.json()** é um método da Fetch API que lê e converte o corpo de uma resposta de forma assíncrona.
 - E) **Object.keys()** extrai os nomes das propriedades de um objeto.
-

ITEM 08

Nível de Dificuldade: Difícil

Objetos de Conhecimento: 8.4 Bibliotecas e APIs

Capacidade: CT06 – Aplicar linguagem de programação por meio de apis, bibliotecas, frameworks na construção de rotinas de software.

Contexto: Um sistema de e-commerce precisa processar pagamentos via cartão de crédito. Como a comunicação com o banco pode demorar vários segundos, o desenvolvedor deve evitar que a interface do usuário congele durante a espera.

Comando: Qual característica do JavaScript permite que tarefas demoradas sejam executadas em segundo plano, mantendo a aplicação responsiva?

Alternativas:

- ☐ A) Execução Síncrona
- ☐ B) Single-threaded Bloqueante
- ☐ C) Assincronismo

- ☐ D) Hoisting Global
- ☐ E) Coerção de Tipos

Gabarito Sinalizado: C

Justificativa: O assincronismo permite delegar tarefas pesadas às Web APIs do navegador, liberando a thread principal para continuar respondendo às interações do usuário (como cliques e rolagens).

Justificativa dos Distratores:

- A) A execução síncrona lê o código linha por linha e trava a próxima instrução até que a atual termine, o que congelaria a tela.
 - B) O JS é *single-threaded*, mas o comportamento bloqueante é justamente o problema que o assincronismo resolve.
 - D) *Hoisting* refere-se à elevação de declarações.
 - E) Coerção de tipos é a conversão automática entre tipos de dados.
-

ITEM 09

Nível de Dificuldade: Médio

Objetos de Conhecimento: 8 Linguagem de programação

Capacidade: CT01 – Reconhecer requisitos de qualidade, integridade, usabilidade e segurança da informação.

Contexto: Em um painel administrativo, o sistema deve exibir mensagens diferentes baseadas em níveis de acesso numéricos (1 para visualização, 2 para edição, 3 para controle total).

Comando: Qual instrução é a mais indicada para rotear caminhos lógicos quando todos dependem da avaliação de uma única expressão contra vários valores exatos?

Alternativas:

- ☐ A) if / else aninhado
- ☐ B) while
- ☐ C) switch
- ☐ D) for
- ☐ E) ternary

Gabarito Sinalizado: C

Justificativa: O **switch** é ideal para roteamento de múltiplos caminhos baseados em uma única variável, utilizando comparação estrita e sendo mais organizado que múltiplos **else if**.

Justificativa dos Distratores:

- A) **if/else** aninhados tornam a auditoria de código difícil e geram a "Opção Pirâmide".
 - B) **while** é para repetição, não para decisão baseada em casos.
 - D) **for** é focado em automação de tarefas repetitivas.
 - E) Ternário é limitado a apenas dois caminhos (verdadeiro/falso).
-

ITEM 10

Nível de Dificuldade: Difícil

Objetos de Conhecimento: 8.3 Boas práticas

Capacidade: CT01 – Reconhecer requisitos de qualidade, integridade, usabilidade e segurança da informação.

Contexto: Durante uma auditoria de código, foi identificado que o uso de `innerHTML` para injetar comentários de usuários estava expondo a plataforma a scripts maliciosos. A equipe deve refatorar o código para garantir a segurança dos dados.

Comando: Caso seja estritamente necessário renderizar HTML rico vindo de uma fonte externa, qual procedimento deve ser adotado antes da injeção no DOM?

Alternativas:

- ☐ A) Usar o método `alert()` para avisar o usuário.
- ☐ B) Converter os dados usando `JSON.stringify()`.
- ☐ C) Passar os dados por uma biblioteca de sanitização, como o `DOMPurify`.
- ☐ D) Trocar o navegador do usuário.
- ☐ E) Usar o operador ternário para ocultar o erro.

Gabarito Sinalizado: C

Justificativa: Quando o `textContent` não é suficiente (pois ele quebraria o design de textos ricos), o dado deve passar por um processo de sanitização para remover partes perigosas do HTML antes de usar o `innerHTML`.

Justificativa dos Distratores:

- A) `alert()` não limpa o código malicioso.
- B) `JSON.stringify()` transforma objetos em texto, não remove scripts de strings HTML.
- D) A vulnerabilidade está no código do site, não no navegador do usuário.
- E) O operador ternário não tem capacidades de filtragem de segurança de strings.